# Speaker Notes

## Face Verification Attendance System

## PRESENTER 1

### 1 Cover — Introduction

"Good morning. We are Group 04 from KDU, Faculty of Computing.

"Our project is the Face Verification Attendance System for the CS31092 module.

"A camera watches the classroom, recognises each student's face using AI, and marks attendance automatically. No paper, no roll call."

### 3 Problem & Solution

"The problem: manual attendance is slow, easy to fake, and unreliable. Students can sign for absent friends, paper gets lost, and there are no real-time records.
"

"Our solution: a camera at the front of the classroom. When a student stands in front of it, the system recognises their face and marks them present in under one second. The lecturer sees everything on a web dashboard.
"

"I will now hand over to Kavishan who will explain exactly how the system processes each camera frame."

### 4 System Architecture

"Our system works in two modes.

"Registration — done once per student. Admin logs in, camera takes a photo, and the face is converted into a fingerprint stored in memory.

"Verification — runs during class. Every camera frame goes through our preprocessing pipeline, then face detection, then face matching. If matched, attendance is logged instantly.

"Now let me explain the preprocessing pipeline — the most important part of our project."

### 5 Preprocessing Overview — All 11 Steps

"Before the AI can recognise any face, the raw camera image must be cleaned and prepared. We have 11 steps. Each one solves a specific real-world problem.

"Without these steps, the system fails in a real classroom — wrong colours, missed faces, too slow. Let me explain each one."

### 6 Steps 1, 2 & 3 — Capture, BGR→RGB, Resize

"Step 1 — Frame Capture. The camera gives us 30 photos every second. Each photo is a grid of colour numbers.
"

"Step 2 — BGR to RGB. This is a very important fix. Our camera library saves colours in the wrong order — Blue, Green, Red. But our AI model expects Red, Green, Blue. If we don't swap them, the AI sees wrong colours and produces the wrong result. One line of code fixes it — but skipping it breaks everything silently.
"

"Step 3 — Resize to 25%. We shrink the image to one quarter of its size. This makes face detection 94% faster so the system can run in real time."

## PRESENTER 2

### 7 Steps 4 & 5 — Grayscale & CLAHE

"Step 4 — Grayscale. We convert to black and white. Our face detector only needs brightness changes, not colour. This makes it 3 times faster.
"

"Step 5 — CLAHE. This is our lighting fix, and one of the most important steps.

"In a lecture hall, students near windows appear dark in the photo. Students under ceiling lights appear washed out. Without fixing this, the face detector misses these faces completely.

"CLAHE divides the image into small 8 by 8 tiles and fixes the brightness of each tile separately. We apply it only to the brightness channel in LAB colour space — so colours and skin tones stay natural."

💡 CLAHE is the step the examiner will ask about most. Key points: (1) 8×8 tiles — local, not global. (2) LAB space — only brightness is fixed, not colours. (3) Without it — dark faces are simply not detected.

### 8 Steps 6, 7 & 8 — Gaussian Blur, HOG, ROI Crop

"Step 6 — Gaussian Blur. Camera sensors add random errors to every pixel. These create fake edges that confuse the face detector. Gaussian Blur smooths them away by averaging each pixel with its neighbours.
"

"Step 7 — HOG Detection. This is how we find where the face is. HOG scans the image looking for patterns of brightness change that match the shape of a face — eye hollows, nose, jawline. When found, it draws a box around the face.
"

"Step 8 — ROI Crop. We cut out just the face. No background, no other students. We add a small 20-pixel border so we don't cut off the chin or forehead."

💡 For HOG — point to your own eyes and jaw while describing. It makes it visual. Keep it brief — one sentence per step is enough here.

## 9 Steps 9, 10 & 11 — Align, Normalise, Encode

"Step 9 — Face Alignment. If a student tilts their head, we rotate the image so both eyes are always horizontal. This gives consistent results.
"

"Step 10 — Normalisation. The AI model expects pixel values in a small range. We scale everything down so the AI gets data in the exact format it was trained on.
"

"Step 11 — ResNet-34 Encoding. This is the final and most powerful step. The cleaned face is passed through a deep neural network with 34 layers, trained on 3 million face photos. It converts the face into exactly 128 numbers — a unique fingerprint. Same person always gives similar numbers. Different people give very different numbers.
"

"I will now hand over to Gajanayaka who will explain how we use these 128 numbers to identify students."

## 10 HOG — 5 Stages (brief)

"Thank you. HOG works in 5 stages — it measures brightness changes, groups them into cells, normalises them, runs a face classifier, and repeats at multiple scales to detect faces of different sizes.

"We chose HOG because it runs at 30 frames per second in real time. CNN detection is more accurate for angled faces but too slow for live attendance."

# PRESENTER 3

## 11 ResNet-34 & Euclidean Matching

"Now — how do we identify who the face belongs to?

"ResNet-34 converts the prepared face into 128 numbers. Think of them as GPS coordinates — same person always lands nearby, different people land far apart.

"We compare these 128 numbers to every registered student's 128 numbers using Euclidean distance — the straight-line mathematical distance.

"If the distance is below 0.50, the student is verified and attendance is marked. Above 0.50, we say Unknown. We chose 0.50 because it balances between being too strict and too lenient."

💡 The GPS analogy is very clear. Use it. If asked about 0.50: 'We tested 0.45 and 0.55 — 0.45 rejected valid students, 0.55 matched similar faces. 0.50 was the best balance.'

## 12 Code & Database

"Our Flask web app has four key functions.

"load_known_faces — reads all student photos at startup and stores their 128-number fingerprints in memory.

"gen_frames — the live video engine. Runs 30 frames per second, applies all preprocessing, does the matching, and streams video to the browser.

"Duplicate prevention — if the same student is logged in the last 60 seconds, we skip. Without this, one student standing still creates hundreds of records.

"MongoDB stores three things per record: student name, subject, and timestamp. The lecturer views all records on the dashboard."

## 13 Progress & Future Work

"We have completed the full core system — all 11 preprocessing steps, face detection, recognition, the web app, student registration, MongoDB logging, duplicate prevention, and the dashboard.
"

"For future work — we plan multi-frame confirmation to reduce false positives, CNN-based detection for better accuracy, Excel export for lecturers, and an anti-spoofing module so a printed photo cannot fool the system."

## 14 Thank You & Closing

"Thank you for your time.

"In short — our system uses an 11-step preprocessing pipeline to prepare each camera frame, ResNet-34 to create a 128-number face fingerprint, and Euclidean distance matching to verify identity. All attendance is logged to MongoDB automatically.

"We are happy to take any questions."

## Likely Questions — Short Answers

Study these. Know the idea — say it in your own words.

**Q Why BGR to RGB?**

OpenCV saves colours in BGR order. Our AI was trained on RGB. Without swapping, the AI sees wrong colours and produces a wrong fingerprint. System runs but identifies nobody — silent failure.

**Q Why resize to 25%?**

Face detection scans every position in the image — too slow at full size. 25% reduces pixels by 94%, making real-time detection possible on a normal laptop.

**Q What is CLAHE?**

It divides the image into 8×8 tiles and fixes brightness of each tile separately. Applied only to the Luminance channel in LAB colour space — only brightness changes, skin tones stay natural. Needed because classroom lighting is uneven.

**Q What are the 128 numbers?**

A mathematical fingerprint of the face. Same person always gives similar numbers. Different people give very different numbers. We compare them using Euclidean distance.

**Q Why 0.50 threshold?**

Tested 0.45 — too strict, rejected valid students. Tested 0.55 — too lenient, matched similar faces. 0.50 was the best balance.

**Q What is HOG?**

Scans the image for patterns of brightness change that match a face shape. Draws a box around the face. Fast enough for 30 FPS on a CPU.

**Q Can a photo fool the system?**

Yes currently — that is a known limitation. We plan an anti-spoofing module as future work to detect 2D vs 3D faces.